ChatGPT

Etiqueta de modelo: (no observada)
Leida: 2026-11-09T21:27:26.353Z
Caracteres: 13823
Fuentes citadas (verificables): 22
Enlaces en el DOM: 22
Fin de respuesta: observado

Pregunta

Contexto: ChatCouncil es una app de investigación que abre diez páginas web reales de proveedores de IA a la vez, cada una en su propio webview nativo con la sesión del usuario; escribe en sus campos de texto, lee sus respuestas del DOM, adjunta archivos y arma un informe. Se está reescribiendo con el núcleo en Rust y un solo código para Windows, macOS, Linux, Android e iOS (escritorio, móvil y tableta), con el rendimiento (RAM, CPU, tiempo de arranque y tamaño) como prioridad absoluta.

Ya medido en Windows, con diez webviews nativos de WebView2 en la misma máquina: Flutter con plugins que dibujan por textura gasta unos 5 puntos de CPU extra cuando las páginas redibujan; Flutter con ventanas nativas y Slint con ventanas nativas (wry) gastan casi cero en el proceso propio; Slint arranca en 0,76 s contra 3,35 s de Flutter, pesa 11 MiB contra 25,5 MiB y usa 74 MB de RAM en el proceso propio contra 122 MB. Se descartaron Dioxus (su interfaz sería un webview más) y Rune (no tiene Android ni iOS).

Pregunta: ¿qué opciones de interfaz y de manejo de varios webviews nativos simultáneos, viables hoy para esas cinco plataformas con un solo código y un núcleo en Rust, no se están considerando, y cómo se comparan en rendimiento medido con Slint y con Flutter? Para cada opción: fuentes verificables con fecha, estado real en cada plataforma (en especial varios webviews nativos simultáneos en Android e iOS), madurez, licencia y riesgos.

Respuesta

Al 9 de noviembre de 2026, la alternativa nueva que probaría primero es Native Shell (longbridge/nsh). Ofrece una interfaz compartida en Rust, usa los motores web del sistema en las cinco plataformas y expone inyección de JavaScript y mensajería con la página. Mantendría Slint como referencia y probaría en paralelo un host nativo mínimo para Android e iOS. Eso permite averiguar si el cuello de botella está en la interfaz o en mantener diez páginas autenticadas, activas y automatizables en un teléfono. La confianza es alta en la cobertura de API documentada y desconocida en el rendimiento de cualquiera de estas rutas con la carga completa de ChatCouncil. Native Shell+4

Interpreto un solo código como una interfaz y un núcleo compartidos, con adaptadores pequeños por plataforma si hacen falta. En móvil, interpreto simultáneos como diez instancias vivas que pueden completar su trabajo mientras la app está en primer plano, aunque solo una esté visible. Si el requisito incluye seguir leyendo el DOM con la app suspendida, iOS cambia el alcance de la solución: UIKit puede suspender el proceso al pasar a segundo plano. Apple Developer Documentation

Opciones y estado real por plataforma

Opción | Windows, macOS y Linux | Android e iOS | Madurez, licencia y riesgo para ChatCouncil |
Native Shell 0.4.0 | WebView2 en un HWND hijo, WKWebView y WebKitGTK 6, respectivamente. En Windows, los diálogos dibujados por la interfaz pueden quedar debajo de la vista web. | android.webkit.WebView y WKWebView dentro de la jerarquía nativa. El componente expone JavaScript y mensajes, suficientes para intentar escribir y leer el DOM. La API de entrada con eventos de confianza solo funciona en macOS en esta versión. No encontré una prueba publicada de diez instancias activas ni de adjuntos automatizados en ambos móviles. | Apache-2.0, versión 4 nov 2026, proyecto joven con backends y ejemplos publicados. Es la vía Rust más directa para probar un reemplazo completo. El primer filtro es que los diez sitios acepten el envío y los adjuntos mediante la API disponible o mediante una extensión nativa pequeña. GitHub+5 |
Host nativo propio + UI compartida | Conservar Wry o alojar directamente WebView2 en Windows y WKWebView en macOS. En Linux, integrar WebKitGTK en un contenedor GTK si se requiere Wayland además de X11. | Una Activity con varios WebView en Android y un controlador UIKit con varios WKWebView en iOS. Android y Apple documentan múltiples instancias y la posible pérdida de renderizadores. Esa capacidad del SO no establece cuántas páginas pesadas se sostienen en un dispositivo concreto. | APIs de plataforma maduras. Wry 0.57.1, MIT o Apache-2.0, 23 oct 2026. La app se hace cargo de foco, teclado, ciclo de vida, perfiles, popups y archivos. Es la ruta con mayor control y la que permite mantener el escritorio ya medido mientras se resuelve móvil. Docs.rs+6 |
Compose Multiplatform + ComposeNativeWebView | WebView2 mediante DirectComposition, WKWebView y WebKitGTK. El plugin actual requiere Nucleus Tao en escritorio. El proyecto documenta que los fotogramas de la página se componen fuera de Compose. | Usa Android WebView y WKWebView. Tiene una suite con hosts reales para ambos y expone evaluación JavaScript. El puente común se inyecta después de la carga en móvil, frente a inicio de documento en escritorio. No encontré una prueba de diez vistas con sesiones y adjuntos. | Plugin Apache-2.0, 1.0.3, 3 sep 2026. Compose para iOS es estable desde mayo de 2025. La UI sería Kotlin y el núcleo puede seguir en Rust vía UniFFI o FFI. El riesgo principal es la dependencia del plugin y de Nucleus Tao, además de la JVM en escritorio. GitHub+5 |
Avalonia 12 + NativeWebView | WebView2, WKWebView y WPE WebKit o WebKitGTK. La ruta WPE en Linux compone fuera de pantalla mediante memoria compartida. La documentación antigua de Avalonia 11 que limitaba Linux a NativeWebDialog corresponde a otra versión. | Controles nativos en ambos móviles y API común para JavaScript y mensajes. La matriz confirma el control embebido. Quedan por comprobar la concurrencia de diez vistas y el flujo de adjuntos. | MIT, incluido el componente web. Avalonia 12 salió el 7 abr 2026. Alternativa viable con UI C#/XAML y núcleo Rust

por FFI. El principal riesgo de rendimiento es trasladar a Linux la hipótesis de bajo coste de composición observada con ventanas hijas en Windows. Avalonia UI Blog+5 |
Uno Platform | WebView2 en Windows, WKWebView en macOS y WebKitGTK alojado en una ventana separada en Linux. La guía indica usar el backend X11 también bajo Wayland. | Android.Webkit.WebView y WKWebView, con acceso documentado al control subyacente. Eso permite extender el wrapper para funciones específicas del sitio. Diez vistas activas y carga de archivos siguen por validar. | Apache-2.0. La versión 6.6, 29 jul 2026, incorporó Native AOT en las cinco plataformas. Es una vía C#/XAML con Rust por FFI. AOT aporta una opción de compilación que vale la pena medir, y la dependencia de X11 pesa si Linux nativo en Wayland es requisito. GitHub+3 |
Qt Quick + Qt WebView + Rust | Windows puede usar WebView2 y macOS el webview del sistema. Linux usa Qt WebEngine, con su propia distribución de Chromium. | Qt WebView usa los webviews de plataforma y expone runJavaScript. Qt documenta restricciones de superposición con QML. No encontré una prueba de diez vistas móviles con automatización y adjuntos, y KDAB indica que las plataformas fuera de su matriz habitual de CXX-Qt reciben menos pruebas. | Qt 6.12 LTS, 30 sep 2026. Qt WebView se ofrece bajo LGPLv3, GPLv2 o licencia comercial. CXX-Qt es MIT o Apache-2.0. Qt es maduro y viable si aceptas otro motor en Linux y una UI QML. El tamaño a medir incluye Qt WebEngine y sus dependencias. qt.io+7 |
BeeWare Toga + Rust por FFI | WebView2 en Windows, widget web en macOS y WebKitGTK con el backend GTK de Linux. Toga advierte que el soporte de WebView en GTK4 es parcial. | Widget web y evaluación JavaScript documentados en ambas plataformas. El issue de carga y descarga de archivos en WebView móvil seguía abierto al consultarlo. | BSD-3-Clause, 0.5.7, 1 oct 2026. Ofrece una sola UI Python con controles de plataforma. La brecha de adjuntos y el coste de distribuir el runtime de Python hacen que tenga prioridad baja para esta app. GitHub+4 |
Hay otras opciones Rust útiles como interfaz, con distinta evidencia sobre el alojamiento web:
Makepad tiene una de las mejores pruebas de que la misma UI Rust se puede distribuir en las cinco plataformas: Robrix publica builds para las tres de escritorio, Android e iOS/iPadOS. makepad-widgets 1.0.0, 13 may 2025, es MIT o Apache-2.0. No encontré una API pública de alojamiento web nativo en las cinco. Su evaluación para ChatCouncil depende del mismo host por plataforma descrito arriba. GitHub+4
egui es la alternativa de UI Rust con mejores comparaciones públicas frente a Slint y Flutter. eframe 0.37.0, 15 oct 2026, MIT o Apache-2.0, documenta Windows, macOS, Linux y Android. iOS queda como integración propia, para la que winit sí ofrece backend UIKit. La ruta de cinco plataformas exige mantener ese host y el alojamiento web. Docs.rs+3
WaterUI merece seguimiento por su WebView con JavaScript, publicado como 0.4.1 el 11 sep 2026. Es MIT o Apache-2.0 y el proyecto se declara pre-1.0. La UI tiene backend Apple, Hydrolysis para Android y Windows, y rutas GTK4/Hydrolysis en Linux. La capa web permite seleccionar motores como CEF o WPE. En Linux, la ruta WPE documenta composición GPU por DMA-BUF sin lectura de vuelta a CPU. La documentación que pude verificar deja abierta la paridad del backend web en Windows y Android para este caso. Docs.rs+5

Fission ofrece UI Rust con GPU para las cinco plataformas, Apache-2.0, con versión 0.15.1 del 5 oct 2026. Documenta módulos nativos para extender el host. No encontré una API pública equivalente a NativeWebView para los cinco destinos, por lo que también depende de una integración propia. GitHub+2

Tauri y Flutter tienen matices importantes como referencias. Wry documenta que build_as_child no está soportado en Android ni iOS. En Tauri, add_child se compila para escritorio con la función unstable, y el soporte móvil de varias ventanas usa Activity Embedding en Android y UIScene en iOS. Eso sirve para ventanas o escenas adicionales, especialmente en tabletas, y deja por resolver diez vistas dentro de la misma pantalla en un teléfono. En Flutter, el plugin oficial cubre Android, iOS y macOS. En Android hay distintas rutas de composición, incluida HCPP experimental desde Flutter 3.44 con Android API 34+, Vulkan e Impeller. Los ~5 puntos de CPU observados en Windows corresponden a la ruta por textura que mediste. Rust+5

Rendimiento medido

La única comparación que encontré con diez páginas reales en el mismo equipo es la que aportaste. En Windows, Slint con Wry dio 0,76 s, 11 MiB y 74 MB en el proceso propio, frente a 3,35 s, 25,5 MiB y 122 MB de Flutter. Con ventanas nativas, ambos tuvieron un coste de CPU en el proceso propio cercano a cero durante el redibujado de las páginas. Falta medir la huella total atribuible a la app, incluidos los procesos de WebView2 y la composición. Microsoft recomienda reutilizar un CoreWebView2Environment entre controles cuando la configuración de sesiones lo permita. GitHub

La mejor comparación pública de arranque y memoria de la interfaz sola que encontré fue guibench, del 23 jul 2026. Usa Arch Linux, i9-12900K y Weston headless. Estas son las cifras de su app mínima, con caché caliente. GitHub+2

UI del benchmark | Arranque externo | RAM en reposo, PSS | Binario medido | Slint 1.17.1 | 51,3 ms | 47,4 MiB | 19,4 MiB |
egui / eframe 0.35.0 | 58,6 ms | 48,4 MiB | 18,8 MiB |
Qt Widgets 6.11.1 | 54,9 ms | 23,6 MiB | 0,2 MiB |
Flutter 3.44.7 | 126,8 ms | 78,2 MiB | 5,1 MiB |
Tauri 2.11.5 | 163,9 ms | 59,0 MiB | 5,3 MiB |

La columna de binario usa distintos perímetros. Qt y Tauri excluyen bibliotecas dinámicas, y Flutter excluye aproximadamente 17 MiB de libflutter. Tampoco es una medición de Qt Quick o de Qt WebEngine. En otro ensayo público de macOS, del 15 ago 2026, una aplicación de cuatro paneles dio picos de 133,0 MB para Slint, 155,3 para egui, 174,0 para Qt, 361,3 para Avalonia, 363,8 para Flutter y 652,6 para Compose Multiplatform/JVM. Ese ensayo mide una interfaz de trabajo distinta, usa otras configuraciones y publica el código, aunque no los registros brutos. GitHub+2

Hay dos mediciones más próximas a las alternativas principales:

Native Shell, informe del fabricante. En Windows, su app Rust mínima registra ~14 MB de memoria privada, ~60 MB de working set y 15,0 MB de ejecutable. En la galería completa, el working set llega a ~390–405 MB después de visitar la sección web y vídeo, junto con otros recursos acumulados. Son dos cargas distintas y ninguna reproduce diez proveedores. Native Shell

ComposeNativeWebView, medición del mantenedor con una vista en un Mac M4. Registra 90 fps frente a 90 fps con Canvas 2D, y 31–34 frente a 32–35 fps con un shader WebGL, comparando la vista embebida con WKWebView directo. Es evidencia sobre esa ruta de composición en macOS. Para Makepad, WaterUI, Fission, Toga, Uno y el host propio, no encontré una comparación pública equivalente con Slint y Flutter para los cuatro indicadores pedidos. GitHub+1

La prueba que decidiría la arquitectura

Probar primero diez vistas en un Android y un iPhone físicos. Empezaría con el host mínimo del SO y repetiría con Native Shell. Usaría diez páginas locales instrumentadas para comprobar concurrencia y luego los diez proveedores reales para verificar inicio de sesión, envío, lectura completa del DOM y adjuntos. Probaría una vista visible y nueve ocultas, cambio de pestaña, presión de memoria y regreso desde segundo plano. WebKit puede reducir o suspender la actividad de páginas inactivas, y Android y Apple tienen callbacks específicos para pérdida del renderizador. Una vista que conserva su sesión pero deja de entregar respuestas cuenta como fallo para esta carga. WebKit+3

Separar adjuntos de ejecución JavaScript. Android expone onShowFileChooser por vista. iOS ofrece el selector por defecto y permite personalizar ese flujo mediante WKUIDelegate desde iOS 18.4. Probaría por separado el usuario selecciona el archivo y ChatCouncil adjunta un archivo que ya tiene autorizado. También comprobaría el login dentro del webview para cada proveedor y método de acceso. Por ejemplo, Google restringe OAuth en navegadores embebidos, de modo que un fallo allí puede persistir aunque cambies de toolkit. developer.android.com+3

Solo después comparar rendimiento con Slint y Flutter. Mediría 0, 1 y 10 vistas, con el mismo motor, versión, perfil, tamaño de vista y carga dentro de cada SO. Separaría primer fotograma de la UI de décima vista lista para ejecutar el adaptador, y registraría RAM y CPU del host y de los renderizadores atribuibles, además del tamaño instalado con sus dependencias. En Android, dumpsys meminfo del paquete por sí solo omite el renderizador aislado. Mantendría una carga reproducible para el A/B y los sitios reales para la prueba de aceptación. developer.android.com+1

La decisión que tomaría hoy es conservar Slint en escritorio y enfrentar Native Shell al host móvil mínimo antes de reescribir la interfaz. Confirmaría Native Shell si completa el flujo con los diez proveedores en ambos móviles y mejora una métrica prioritaria a igualdad de carga. Si falla un adjunto, una sesión o el progreso de una vista oculta, el host mínimo permite localizar si el límite viene del wrapper, del SO o del sitio. El coste de decidir con una demo de una vista sería descubrir esa diferencia después de migrar la UI. La revisión verificó documentación, código publicado y mediciones ajenas. No ejecuté ChatCouncil ni reproduje la prueba en dispositivos físicos.

Texto capturado tal cual de la interfaz del proveedor, sin corregir ni recortar.
Este documento es la respuesta a la pregunta, no la evaluacion que este
proveedor hizo de las respuestas de los demas.